FLOWFORGE

Visual pipelines. Inspectable code.

```
> BUILD    Connect transformations on a graph
> RUN      Preview each step in DuckDB
> EXPORT   SQL, PySpark and a dbt project
```

12 STEP TYPES 4 CODE TARGETS

[01] WHY A VISUAL WORKSTATION

THE PROBLEM

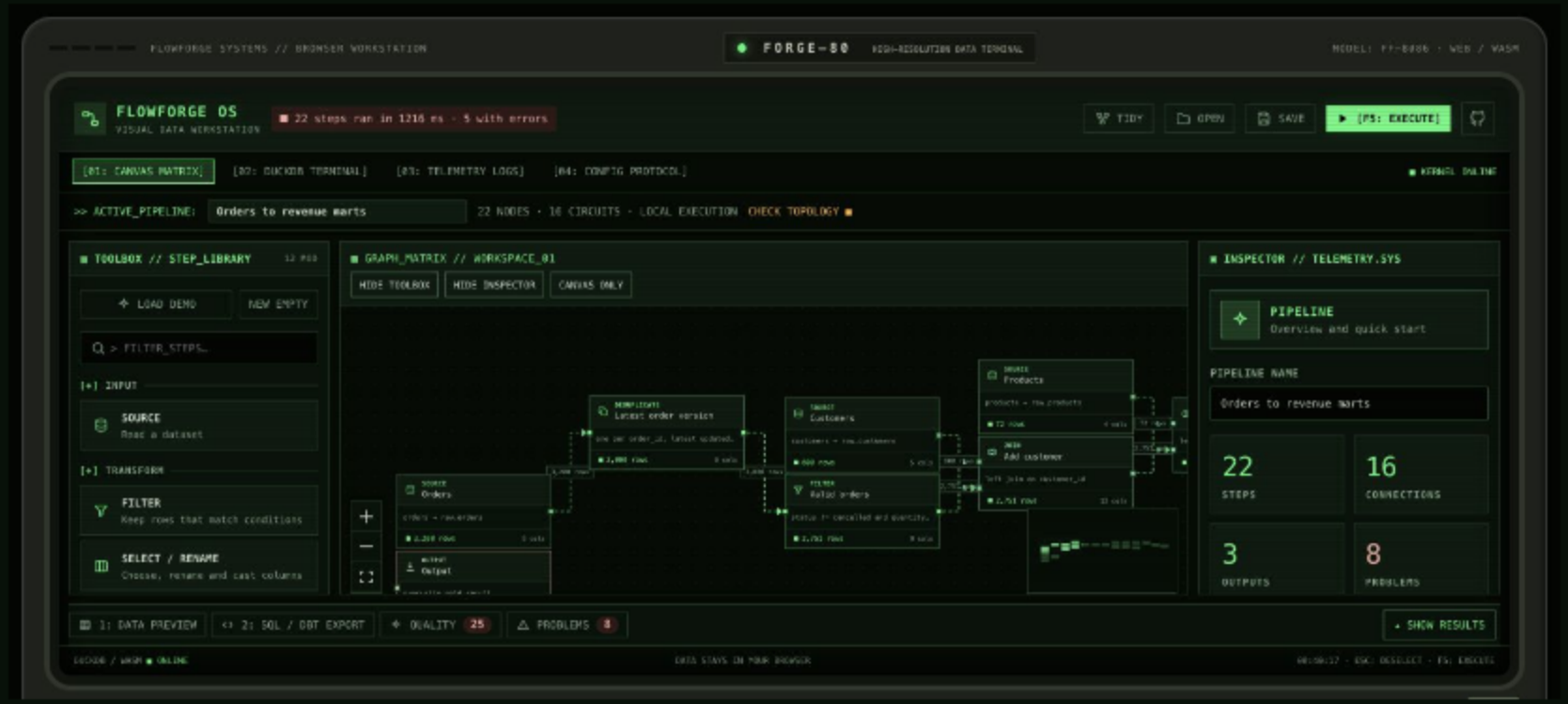
A pipeline diagram alone cannot show what a join changed or what code will run after export.

THE IMPLEMENTATION

A typed graph drives local execution. Each node exposes its schema and preview, while exports remain readable.

Design goal: shorten the loop between graph, data and code.

[02] THE GRAPH MAKES ERRORS VISIBLE

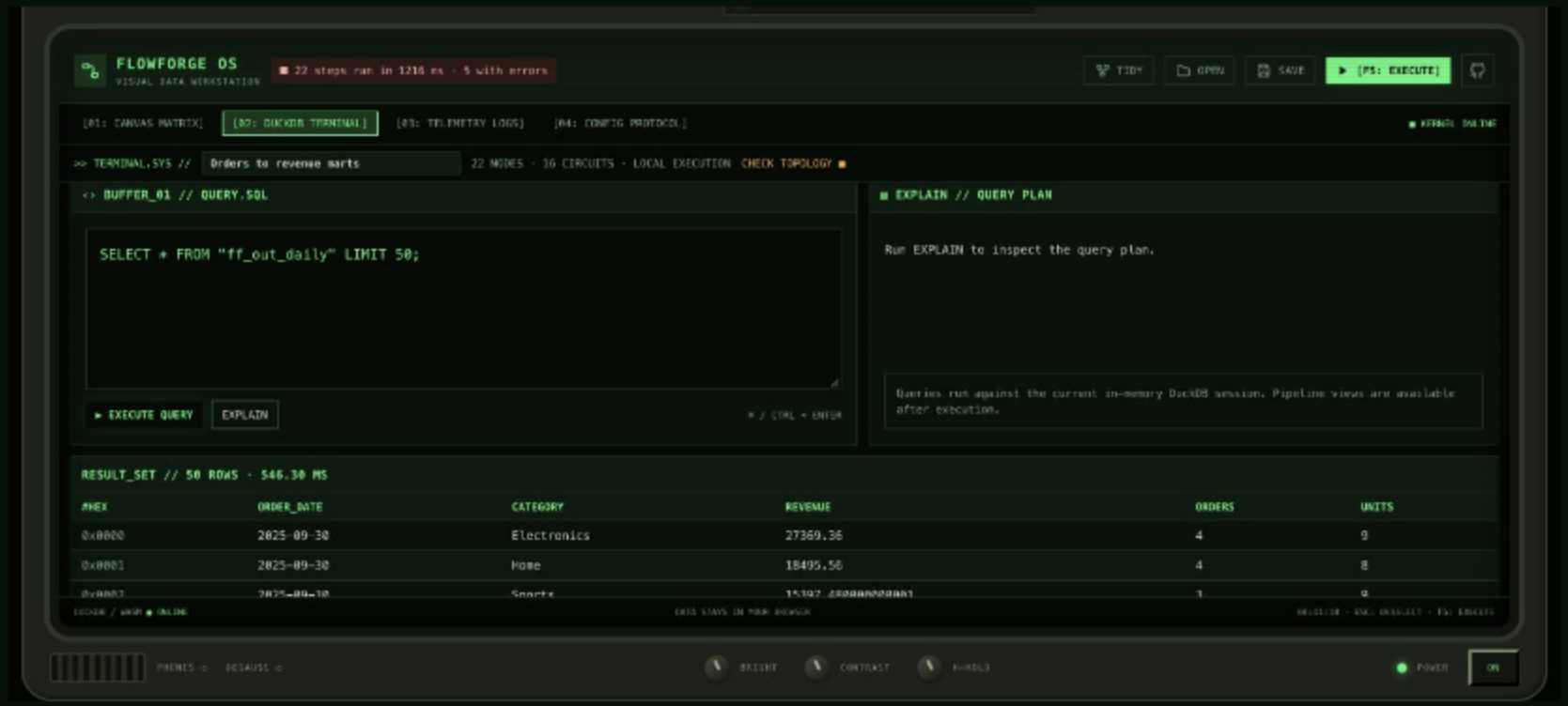


Current workspace: disconnected steps trigger topology warnings.
Valid branches still expose row counts and schemas.

[03] COMPILER AND EXECUTION DESIGN

01	**GRAPH**	Typed step configuration and named input ports
02	**VALIDATION**	Required inputs, invalid configuration and cycle checks
03	**EXECUTION**	Dependency order with a DuckDB view for each step
04	**EXPORT**	Shared SELECT compiler for SQL targets; separate PySpark emitter

[04] SQL REMAINS AVAILABLE



The terminal queries the same in-memory views as the graph.
This keeps the transformation logic open to inspection.

[05] VALIDATION AND LESSONS

17

native-engine tests
reported in the repository

Exported SQL: end-to-end DuckDB checks
PySpark: Python AST syntax checks
Joins: explicit, collision-free columns

LIMITS

AST checks do not prove Spark runtime parity. Expressions need target compatibility. Browser memory limits scale.

adityamalviya04692.github.io/flowforge/